

Thiết kế ApiResponse chuẩn Production cho Spring Boot

1. Mục tiêu

Trong hệ thống production, response API cần được chuẩn hóa để:

- Frontend/mobile dễ parse dữ liệu.
- Backend dễ maintain và mở rộng.
- Có format thống nhất giữa các API.
- Có `traceId` để debug production.
- Có `code/message` rõ ràng cho business.
- Hỗ trợ pagination, metadata, error response.
- Không expose thông tin nhạy cảm hoặc internal implementation.

Một response API production thường cần trả lời được các câu hỏi:

- Request thành công hay thất bại?
- Mã xử lý nghiệp vụ là gì?
- Message hiển thị cho client là gì?
- Data chính nằm ở đâu?
- Có metadata như paging, sorting, traceId không?
- Nếu lỗi thì field nào lỗi, lý do gì?

2. Nguyên tắc thiết kế ApiResponse

2.1. Response thành công và response lỗi nên tách rõ

Không nên dùng một DTO quá chung cho mọi thứ nếu làm response bị rối.

Có thể thiết kế:

```
ApiResponse<T>      -> dùng cho success response
ErrorResponse       -> dùng cho error response
PagedResponse<T>   -> dùng cho response phân trang
```

Ví dụ:

```
{
  "success": true,
```

```
"code": "SUCCESS",
"message": "Request processed successfully",
"data": {
  "id": 1001,
  "status": "PAID"
},
"traceId": "8d2f7a98b3c94d22",
"timestamp": "2026-06-04T10:00:00Z"
}
```

Error response nên tách riêng:

```
{
  "success": false,
  "status": 400,
  "error": "BAD_REQUEST",
  "code": "ORDER_001",
  "message": "Order has already been paid",
  "path": "/api/orders/1001/pay",
  "traceId": "8d2f7a98b3c94d22",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

2.2. Không nên trả entity trực tiếp

Không nên:

```
@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {
    return userService.getUser(id);
}
```

Vì có thể expose field nhạy cảm hoặc làm response phụ thuộc DB entity.

Nên:

```
@GetMapping("/{id}")
public ResponseEntity<ApiResponse<UserResponse>> getUser(@PathVariable Long id)
{
    UserResponse response = userService.getUser(id);
}
```

```
    return ResponseEntity.ok(ApiResponse.success(response));  
}
```

2.3. Không nên trả message lung tung ở từng API

Không nên mỗi API tự định nghĩa message khác nhau:

```
{  
  "msg": "OK"  
}
```

```
{  
  "message": "success"  
}
```

```
{  
  "status": "done"  
}
```

Nên thống nhất:

```
{  
  "success": true,  
  "code": "SUCCESS",  
  "message": "Request processed successfully",  
  "data": {}  
}
```

3. Format ApiResponse đề xuất

3.1. Success response cơ bản

```
{  
  "success": true,  
  "code": "SUCCESS",  
  "message": "Request processed successfully",  
  "data": {  
    // ...  
  }  
}
```

```
{
  "id": 1,
  "name": "iPhone 15"
},
{
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

3.2. Response tạo mới resource

```
{
  "success": true,
  "code": "CREATED",
  "message": "Resource created successfully",
  "data": {
    "id": 1001
  },
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

HTTP status nên là `201 Created`.

3.3. Response update/delete

Update:

```
{
  "success": true,
  "code": "UPDATED",
  "message": "Resource updated successfully",
  "data": {
    "id": 1001,
    "status": "UPDATED"
  },
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Delete có 2 cách:

Cách 1: trả `204 No Content`

HTTP 204
No response body

Cách 2: trả wrapper nếu frontend cần message:

```
{
  "success": true,
  "code": "DELETED",
  "message": "Resource deleted successfully",
  "data": null,
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Trong dự án enterprise, nếu frontend/mobile cần response thống nhất thì thường dùng cách 2. Nếu API public REST strict hơn thì có thể dùng `204 No Content`.

4. ApiResponse class

4.1. Version dùng Java class

```
package com.example.common.response;

import com.fasterxml.jackson.annotation.JsonInclude;
import org.slf4j.MDC;

import java.time.Instant;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ApiResponse<T> {

    private boolean success;
    private String code;
    private String message;
    private T data;
    private Object meta;
    private String traceId;
    private Instant timestamp;

    private ApiResponse(
        boolean success,
        String code,
```

```

        String message,
        T data,
        Object meta
    ) {
        this.success = success;
        this.code = code;
        this.message = message;
        this.data = data;
        this.meta = meta;
        this.traceId = MDC.get("traceId");
        this.timestamp = Instant.now();
    }

    public static <T> ApiResponse<T> success(T data) {
        return new ApiResponse<>(
            true,
            ResponseCode.SUCCESS.getCode(),
            ResponseCode.SUCCESS.getMessage(),
            data,
            null
        );
    }

    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(
            true,
            ResponseCode.SUCCESS.getCode(),
            message,
            data,
            null
        );
    }

    public static <T> ApiResponse<T> created(T data) {
        return new ApiResponse<>(
            true,
            ResponseCode.CREATED.getCode(),
            ResponseCode.CREATED.getMessage(),
            data,
            null
        );
    }

    public static <T> ApiResponse<T> updated(T data) {
        return new ApiResponse<>(
            true,
            ResponseCode.UPDATED.getCode(),
            ResponseCode.UPDATED.getMessage(),

```

```

        data,
        null
    );
}

public static <T> ApiResponse<T> deleted() {
    return new ApiResponse<>(
        true,
        ResponseCode.DELETED.getCode(),
        ResponseCode.DELETED.getMessage(),
        null,
        null
    );
}

public static <T> ApiResponse<T> withMeta(T data, Object meta) {
    return new ApiResponse<>(
        true,
        ResponseCode.SUCCESS.getCode(),
        ResponseCode.SUCCESS.getMessage(),
        data,
        meta
    );
}

public boolean isSuccess() {
    return success;
}

public String getCode() {
    return code;
}

public String getMessage() {
    return message;
}

public T getData() {
    return data;
}

public Object getMeta() {
    return meta;
}

public String getTraceId() {
    return traceId;
}

```

```
    public Instant getTimestamp() {  
        return timestamp;  
    }  
}
```

5. ResponseCode enum

```
package com.example.common.response;  
  
public enum ResponseCode {  
  
    SUCCESS("SUCCESS", "Request processed successfully"),  
    CREATED("CREATED", "Resource created successfully"),  
    UPDATED("UPDATED", "Resource updated successfully"),  
    DELETED("DELETED", "Resource deleted successfully"),  
    ACCEPTED("ACCEPTED", "Request accepted for processing");  
  
    private final String code;  
    private final String message;  
  
    ResponseCode(String code, String message) {  
        this.code = code;  
        this.message = message;  
    }  
  
    public String getCode() {  
        return code;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
}
```

6. PagedResponse chuẩn production

Với API list/search, không nên trả trực tiếp `Page<Entity>` của Spring Data vì nó expose nhiều thông tin không cần thiết.

Không nên:


```
@GetMapping
public Page<Product> getProducts(Pageable pageable) {
    return productRepository.findAll(pageable);
}
```

Nên map về response riêng.

6.1. PageMeta

```
package com.example.common.response;

public class PageMeta {

    private int page;
    private int size;
    private long totalElements;
    private int totalPages;
    private boolean first;
    private boolean last;
    private boolean hasNext;
    private boolean hasPrevious;

    public PageMeta(
        int page,
        int size,
        long totalElements,
        int totalPages,
        boolean first,
        boolean last,
        boolean hasNext,
        boolean hasPrevious
    ) {
        this.page = page;
        this.size = size;
        this.totalElements = totalElements;
        this.totalPages = totalPages;
        this.first = first;
        this.last = last;
        this.hasNext = hasNext;
        this.hasPrevious = hasPrevious;
    }

    public int getPage() {
        return page;
    }
}
```

```
    }

    public int getSize() {
        return size;
    }

    public long getTotalElements() {
        return totalElements;
    }

    public int getTotalPages() {
        return totalPages;
    }

    public boolean isFirst() {
        return first;
    }

    public boolean isLast() {
        return last;
    }

    public boolean isHasNext() {
        return hasNext;
    }

    public boolean isHasPrevious() {
        return hasPrevious;
    }
}
```

6.2. PagedResponse

```
package com.example.common.response;

import org.springframework.data.domain.Page;

import java.util.List;

public class PagedResponse<T> {

    private List<T> items;
    private PageMeta page;
```

```

public PagedResponse(List<T> items, PageMeta page) {
    this.items = items;
    this.page = page;
}

public static <T> PagedResponse<T> from(Page<T> pageData) {
    PageMeta meta = new PageMeta(
        pageData.getNumber(),
        pageData.getSize(),
        pageData.getTotalElements(),
        pageData.getTotalPages(),
        pageData.isFirst(),
        pageData.isLast(),
        pageData.hasNext(),
        pageData.hasPrevious()
    );

    return new PagedResponse<>(pageData.getContent(), meta);
}

public List<T> getItems() {
    return items;
}

public PageMeta getPage() {
    return page;
}
}

```

6.3. Response phân trang

```

{
  "success": true,
  "code": "SUCCESS",
  "message": "Request processed successfully",
  "data": {
    "items": [
      {
        "id": 1,
        "name": "iPhone 15"
      },
      {
        "id": 2,
        "name": "Samsung S24"
      }
    ]
  }
}

```

```

    }
  ],
  "page": {
    "page": 0,
    "size": 20,
    "totalElements": 100,
    "totalPages": 5,
    "first": true,
    "last": false,
    "hasNext": true,
    "hasPrevious": false
  }
},
"traceId": "c5a1f2e9d7b84a3d",
"timestamp": "2026-06-04T10:00:00Z"
}

```

7. Ví dụ Controller sử dụng ApiResponse

7.1. Create API

```

@PostMapping
public ResponseEntity<ApiResponse<OrderResponse>> createOrder(
    @Valid @RequestBody CreateOrderRequest request
) {
    OrderResponse response = orderService.createOrder(request);

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(ApiResponse.created(response));
}

```

7.2. Get detail API

```

@GetMapping("/{id}")
public ResponseEntity<ApiResponse<OrderResponse>> getOrder(
    @PathVariable Long id
) {
    OrderResponse response = orderService.getOrder(id);
}

```

```
        return ResponseEntity.ok(ApiResponse.success(response));
    }
```

7.3. Update API

```
@PutMapping("/{id}")
public ResponseEntity<ApiResponse<OrderResponse>> updateOrder(
    @PathVariable Long id,
    @Valid @RequestBody UpdateOrderRequest request
) {
    OrderResponse response = orderService.updateOrder(id, request);

    return ResponseEntity.ok(ApiResponse.updated(response));
}
```

7.4. Delete API

```
@DeleteMapping("/{id}")
public ResponseEntity<ApiResponse<Void>> deleteOrder(
    @PathVariable Long id
) {
    orderService.deleteOrder(id);

    return ResponseEntity.ok(ApiResponse.deleted());
}
```

Hoặc nếu muốn chuẩn REST strict:

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteOrder(
    @PathVariable Long id
) {
    orderService.deleteOrder(id);

    return ResponseEntity.noContent().build();
}
```

7.5. Search API có phân trang

```
@GetMapping
public ResponseEntity<ApiResponse<PagedResponse<OrderResponse>>> searchOrders(
    @RequestParam(required = false) String keyword,
    Pageable pageable
) {
    Page<OrderResponse> page = orderService.searchOrders(keyword, pageable);

    PagedResponse<OrderResponse> response = PagedResponse.from(page);

    return ResponseEntity.ok(ApiResponse.success(response));
}
```

8. Có nên auto wrap response bằng ResponseBodyAdvice không?

Có thể dùng `ResponseBodyAdvice` để tự động bọc mọi response vào `ApiResponse`.

Ví dụ:

```
@RestControllerAdvice
public class ApiResponseAdvice implements ResponseBodyAdvice<Object> {

    @Override
    public boolean supports(
        MethodParameter returnType,
        Class<? extends HttpMessageConverter<?>> converterType
    ) {
        return true;
    }

    @Override
    public Object beforeBodyWrite(
        Object body,
        MethodParameter returnType,
        MediaType selectedContentType,
        Class<? extends HttpMessageConverter<?>> selectedConverterType,
        ServerHttpRequest request,
        ServerHttpResponse response
    ) {
        if (body instanceof ApiResponse<?>) {

```

```

        return body;
    }

    if (body instanceof ErrorResponse) {
        return body;
    }

    if (body instanceof String) {
        return body;
    }

    return ApiResponse.success(body);
}
}

```

Tuy nhiên, trong production cần cẩn thận vì auto wrap có thể gây lỗi với:

- API trả `String`
- File download
- Streaming response
- Swagger/OpenAPI
- Actuator endpoints
- Spring Security error response
- Response đã được custom format
- Binary response như PDF, Excel, image

Vì vậy, với dự án vừa và nhỏ, nên explicit trong controller:

```
return ResponseEntity.ok(ApiResponse.success(data));
```

Với dự án lớn, có thể dùng `ResponseBodyAdvice` nhưng phải exclude các case đặc biệt.

9. Khi nào không nên dùng ApiResponse wrapper?

Không phải API nào cũng nên bọc bằng `ApiResponse`.

Nên tránh wrapper với:

9.1. File download

```

@GetMapping("/invoices/{id}/download")
public ResponseEntity<Resource> downloadInvoice(@PathVariable Long id) {
    Resource file = invoiceService.download(id);
}

```

```
        return ResponseEntity.ok()
            .header(HttpHeaders.CONTENT_DISPOSITION, "attachment;
filename=invoice.pdf")
            .contentType(MediaType.APPLICATION_PDF)
            .body(file);
    }
```

Không nên:

```
{
  "success": true,
  "data": "binary file here"
}
```

9.2. Streaming API

Ví dụ:

- Server-Sent Events
- video streaming
- large file streaming

Không nên wrap.

9.3. Health check

```
GET /actuator/health
```

Nên giữ format mặc định của Spring Actuator.

9.4. Public API theo chuẩn đối tác

Nếu API phải tuân theo contract của bên thứ ba, không nên tự ý wrap.

10. ErrorResponse nên thiết kế riêng

ApiResponse dùng cho success. ErrorResponse dùng cho failure.


```

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private boolean success;
    private Instant timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private String path;
    private String traceId;
    private List<FieldErrorResponse> details;

    public static ErrorResponse of(
        int status,
        String error,
        String code,
        String message,
        String path,
        String traceId
    ) {
        ErrorResponse response = new ErrorResponse();
        response.success = false;
        response.timestamp = Instant.now();
        response.status = status;
        response.error = error;
        response.code = code;
        response.message = message;
        response.path = path;
        response.traceId = traceId;
        return response;
    }

    public static ErrorResponse validation(
        int status,
        String error,
        String code,
        String message,
        String path,
        String traceId,
        List<FieldErrorResponse> details
    ) {
        ErrorResponse response = of(status, error, code, message, path,
        traceId);
        response.details = details;
        return response;
    }
}

```

```
// getter/setter  
}
```

Ví dụ validation error:

```
{  
  "success": false,  
  "timestamp": "2026-06-04T10:10:00Z",  
  "status": 422,  
  "error": "UNPROCESSABLE_ENTITY",  
  "code": "COMMON_422",  
  "message": "Validation failed",  
  "path": "/api/orders",  
  "traceId": "af29d7b56e1c48a9",  
  "details": [  
    {  
      "field": "quantity",  
      "message": "must be greater than 0",  
      "rejectedValue": -1  
    }  
  ]  
}
```

11. TraceId trong ApiResponse

Production nên có `traceId` trong cả response thành công và lỗi.

Lý do:

- User báo lỗi có thể gửi traceId cho support.
- Backend tìm log nhanh hơn.
- Dễ correlate log giữa API Gateway, service A, service B.
- Dùng tốt với ELK, Grafana Loki, Datadog, New Relic, OpenTelemetry.

Ví dụ filter:

```
@Component  
public class RequestIdFilter extends OncePerRequestFilter {  
  
    private static final String TRACE_ID = "traceId";  
    private static final String TRACE_ID_HEADER = "X-Trace-Id";  

```

```

@Override
protected void doFilterInternal(
    HttpServletRequest request,
    HttpServletResponse response,
    FilterChain filterChain
) throws ServletException, IOException {

    String traceId = request.getHeader	TRACE_ID_HEADER);

    if (traceId == null || traceId.isBlank()) {
        traceId = UUID.randomUUID().toString().replace("-", "");
    }

    try {
        MDC.put	TRACE_ID, traceId);
        response.setHeader	TRACE_ID_HEADER, traceId);
        filterChain.doFilter(request, response);
    } finally {
        MDC.remove	TRACE_ID);
    }
}
}

```

12. Có nên có field `success` không?

Có hai quan điểm.

Cách 1: Có `success`

```

{
  "success": true,
  "code": "SUCCESS",
  "data": {}
}

```

Ưu điểm:

- Frontend dễ check.
- Mobile app dễ xử lý.
- Hữu ích khi HTTP status bị proxy/gateway làm thay đổi.
- Dễ thống nhất với error response.

Nhược điểm:

- Có thể bị trùng ý nghĩa với HTTP status.

Cách 2: Không có `success`

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {}
}
```

Ưu điểm:

- Gọn hơn.
- RESTful hơn.

Nhược điểm:

- Frontend phải dựa vào HTTP status.

Trong dự án enterprise, mình khuyến nghị vẫn nên có `success`, nhất là khi frontend/mobile cần xử lý thống nhất.

13. Có nên có field `code` không?

Nên có.

HTTP status chỉ nói lỗi kỹ thuật ở mức protocol, còn `code` nói lỗi nghiệp vụ cụ thể.

Ví dụ cùng HTTP 409:

```
{
  "status": 409,
  "code": "ORDER_ALREADY_PAID"
}
```

```
{
  "status": 409,
  "code": "COUPON_ALREADY_USED"
}
```

```
{
  "status": 409,
  "code": "VERSION_CONFLICT"
}
```

Frontend có thể dựa vào `code` để:

- Hiển thị message khác nhau.
- Điều hướng flow.
- Show popup cụ thể.
- Translate message đa ngôn ngữ.
- Tracking analytics.

14. Có nên trả message tiếng Việt không?

Tùy hệ thống.

Option 1: Backend trả message cố định

```
{
  "message": "Đơn hàng đã được thanh toán"
}
```

Phù hợp với hệ thống nhỏ, không đa ngôn ngữ.

Option 2: Backend trả code, frontend translate

```
{
  "code": "ORDER_ALREADY_PAID",
  "message": "Order has already been paid"
}
```

Frontend dùng `code` để map sang ngôn ngữ người dùng.

Phù hợp với hệ thống lớn, đa quốc gia, mobile app.

Khuyến nghị production:

- Backend trả message tiếng Anh trung lập.
- Frontend có thể translate theo `code`.
- Với admin/internal tool có thể dùng message từ backend.

15. Có nên để `data: null` không?

Có thể.

Ví dụ:

```
{
  "success": true,
  "code": "DELETED",
  "message": "Resource deleted successfully",
  "data": null
}
```

Hoặc ẩn field null bằng:

```
@JsonInclude(JsonInclude.Include.NON_NULL)
```

Khi đó response sẽ là:

```
{
  "success": true,
  "code": "DELETED",
  "message": "Resource deleted successfully",
  "traceId": "abc123",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Khuyến nghị:

- Nếu muốn response gọn: dùng `NON_NULL`.
- Nếu frontend muốn schema cố định: luôn giữ `data`, kể cả `null`.

16. Có nên để timestamp trong success response không?

Có thể có hoặc không.

Nên có nếu:

- API cần audit.
- Mobile cần debug.
- Hệ thống distributed.
- Có support production thường xuyên.
- Có nhiều timezone.

Không bắt buộc nếu muốn response ngắn.

Khuyến nghị production enterprise:

```
"timestamp": "2026-06-04T10:00:00Z"
```

Nên dùng UTC với `Instant`.

17. Có nên đưa HTTP status vào ApiResponse success không?

Không bắt buộc.

Success response có thể không cần:

```
{
  "success": true,
  "code": "SUCCESS",
  "message": "OK",
  "data": {}
}
```

Vì HTTP status đã nằm ở response status.

Error response nên có `status` vì useful cho debug:

```
{
  "success": false,
  "status": 404,
  "code": "ORDER_NOT_FOUND"
}
```

Khuyến nghị:

- Success: không cần `status`.
- Error: nên có `status`.

18. Response cho async command

Một số API không xử lý xong ngay, chỉ nhận request để xử lý sau.

Ví dụ:

```
POST /api/reports/export
```

Response nên là `202 Accepted`:

```
{
  "success": true,
  "code": "ACCEPTED",
  "message": "Request accepted for processing",
  "data": {
    "jobId": "export-20260604-001",
    "status": "PENDING"
  },
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Controller:

```
@PostMapping("/reports/export")
public ResponseEntity<ApiResponse<ExportJobResponse>> exportReport(
    @Valid @RequestBody ExportReportRequest request
) {
    ExportJobResponse response = reportService.createExportJob(request);

    return ResponseEntity
        .status(HttpStatus.ACCEPTED)
        .body(ApiResponse.accepted(response));
}
```

Nếu cần thêm factory method:


```

public static <T> ApiResponse<T> accepted(T data) {
    return new ApiResponse<> (
        true,
        ResponseCode.ACCEPTED.getCode(),
        ResponseCode.ACCEPTED.getMessage(),
        data,
        null
    );
}

```

19. Response cho batch API

Ví dụ API import nhiều record.

Không nên chỉ trả:

```

{
  "success": true
}

```

Nên trả summary rõ ràng:

```

{
  "success": true,
  "code": "SUCCESS",
  "message": "Import completed",
  "data": {
    "total": 100,
    "successCount": 95,
    "failedCount": 5,
    "failedItems": [
      {
        "row": 10,
        "reason": "Invalid email"
      },
      {
        "row": 25,
        "reason": "Duplicate phone number"
      }
    ]
  },
  "traceId": "c5a1f2e9d7b84a3d",
}

```

```
"timestamp": "2026-06-04T10:00:00Z"
}
```

20. Response cho command không cần data

Ví dụ API change password:

```
{
  "success": true,
  "code": "SUCCESS",
  "message": "Password changed successfully",
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Controller:

```
@PostMapping("/change-password")
public ResponseEntity<ApiResponse<Void>> changePassword(
    @Valid @RequestBody ChangePasswordRequest request
) {
    userService.changePassword(request);

    return ResponseEntity.ok(
        ApiResponse.success("Password changed successfully", null)
    );
}
```

21. Response cho ID-only create

Nhiều API create không cần trả toàn bộ object, chỉ cần trả ID.

```
public class IdResponse {

    private Long id;

    public IdResponse(Long id) {
        this.id = id;
    }
}
```

```
public Long getId() {  
    return id;  
}  
}
```

Response:

```
{  
  "success": true,  
  "code": "CREATED",  
  "message": "Resource created successfully",  
  "data": {  
    "id": 1001  
  },  
  "traceId": "c5a1f2e9d7b84a3d",  
  "timestamp": "2026-06-04T10:00:00Z"  
}
```

22. Response cho API login

Login response nên rõ ràng, nhưng không expose thông tin nhạy cảm.

```
{  
  "success": true,  
  "code": "SUCCESS",  
  "message": "Login successfully",  
  "data": {  
    "accessToken": "jwt-access-token",  
    "refreshToken": "jwt-refresh-token",  
    "tokenType": "Bearer",  
    "expiresIn": 3600,  
    "user": {  
      "id": 1,  
      "email": "user@example.com",  
      "roles": ["USER"]  
    }  
  },  
  "traceId": "c5a1f2e9d7b84a3d",  
  "timestamp": "2026-06-04T10:00:00Z"  
}
```

Lưu ý:

- Không trả password hash.
- Không trả secret key.
- Không log token.
- Nếu dùng cookie httpOnly thì có thể không trả refresh token trong body.

23. Response cho API upload file

```
{
  "success": true,
  "code": "SUCCESS",
  "message": "File uploaded successfully",
  "data": {
    "fileId": "file_123",
    "fileName": "invoice.pdf",
    "contentType": "application/pdf",
    "size": 204800,
    "url": "https://cdn.example.com/files/file_123"
  },
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

24. DTO response không nên chứa field nhạy cảm

Không nên:

```
public class UserResponse {
    private Long id;
    private String email;
    private String password;
    private String refreshToken;
}
```

Nên:

```
public class UserResponse {
    private Long id;
    private String email;
}
```

```
private String fullName;
private List<String> roles;
}
```

25. ApiResponse với Lombok

Nếu dự án dùng Lombok:

```
@Getter
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ApiResponse<T> {

    private final boolean success;
    private final String code;
    private final String message;
    private final T data;
    private final Object meta;
    private final String traceId;
    private final Instant timestamp;

    private ApiResponse(
        boolean success,
        String code,
        String message,
        T data,
        Object meta
    ) {
        this.success = success;
        this.code = code;
        this.message = message;
        this.data = data;
        this.meta = meta;
        this.traceId = MDC.get("traceId");
        this.timestamp = Instant.now();
    }

    public static <T> ApiResponse<T> success(T data) {
        return new ApiResponse<>(
            true,
            "SUCCESS",
            "Request processed successfully",
            data,
            null
        );
    }
}
```

```

    }

    public static <T> ApiResponse<T> success(String message, T data) {
        return new ApiResponse<>(
            true,
            "SUCCESS",
            message,
            data,
            null
        );
    }

    public static <T> ApiResponse<T> created(T data) {
        return new ApiResponse<>(
            true,
            "CREATED",
            "Resource created successfully",
            data,
            null
        );
    }

    public static <T> ApiResponse<T> withMeta(T data, Object meta) {
        return new ApiResponse<>(
            true,
            "SUCCESS",
            "Request processed successfully",
            data,
            meta
        );
    }
}

```

26. ApiResponse với Java record

Nếu dự án dùng Java 17+, có thể dùng `record`.

```

@JsonInclude(JsonInclude.Include.NON_NULL)
public record ApiResponse<T>(
    boolean success,
    String code,
    String message,
    T data,
    Object meta,

```

```

        String traceId,
        Instant timestamp
    ) {

        public static <T> ApiResponse<T> success(T data) {
            return new ApiResponse<>(
                true,
                "SUCCESS",
                "Request processed successfully",
                data,
                null,
                MDC.get("traceId"),
                Instant.now()
            );
        }

        public static <T> ApiResponse<T> created(T data) {
            return new ApiResponse<>(
                true,
                "CREATED",
                "Resource created successfully",
                data,
                null,
                MDC.get("traceId"),
                Instant.now()
            );
        }

        public static <T> ApiResponse<T> withMeta(T data, Object meta) {
            return new ApiResponse<>(
                true,
                "SUCCESS",
                "Request processed successfully",
                data,
                meta,
                MDC.get("traceId"),
                Instant.now()
            );
        }
    }
}

```

Record gọn, immutable, phù hợp với DTO response.

27. Chuẩn package đề xuất

```
src/main/java/com/example/project
├── common
│   ├── response
│   │   ├── ApiResponse.java
│   │   ├── ErrorResponse.java
│   │   ├── FieldErrorResponse.java
│   │   ├── PagedResponse.java
│   │   ├── PageMeta.java
│   │   ├── ResponseCode.java
│   │   └── IdResponse.java
│   ├── exception
│   │   ├── ErrorCode.java
│   │   ├── BaseException.java
│   │   └── GlobalExceptionHandler.java
│   └── tracing
│       └── RequestIdFilter.java
├── order
│   ├── controller
│   ├── service
│   ├── dto
│   │   ├── request
│   │   └── response
│   └── exception
└── payment
    ├── controller
    ├── service
    └── dto
```

28. Quy ước HTTP Status + ApiResponse

Case	HTTP Status	ApiResponse code
Get success	200	SUCCESS
Create success	201	CREATED
Update success	200	UPDATED

Case	HTTP Status	ApiResponse code
Delete success with body	200	DELETED
Delete no body	204	N/A
Async accepted	202	ACCEPTED
Validation fail	400 hoặc 422	COMMON_400 / COMMON_422
Unauthorized	401	COMMON_401
Forbidden	403	COMMON_403
Not found	404	RESOURCE_NOT_FOUND
Conflict	409	CONFLICT
Internal error	500	INTERNAL_SERVER_ERROR

29. Best practices

29.1. Nên làm

- Dùng DTO response riêng, không trả entity.
- Có format response thống nhất.
- Có `traceId`.
- Có `code` rõ ràng.
- Có `message` an toàn.
- Có `PagedResponse` riêng cho phân trang.
- Có `ErrorResponse` riêng cho lỗi.
- Dùng `@JsonInclude(JsonInclude.Include.NON_NULL)` nếu muốn response gọn.
- Dùng `Instant` cho timestamp.
- Không wrap file download, streaming, actuator.

29.2. Không nên làm

Không nên trả response kiểu mỗi API một format:

```
{
  "result": {}
}
```

```
{
  "data": {},
  "status": "ok"
}
```

```
{
  "msg": "done"
}
```

Không nên trả JPA entity:

```
return userRepository.findAll();
```

Không nên expose internal error:

```
{
  "message": "could not execute statement; SQL [n/a]; constraint
[uk_user_email]"
}
```

Không nên dùng HTTP 200 cho mọi lỗi:

```
{
  "success": false,
  "code": "ORDER_NOT_FOUND"
}
```

với HTTP status vẫn là 200.

Production nên dùng đúng HTTP status.

30. Câu trả lời phỏng vấn mẫu

Trong dự án production, em thường chuẩn hóa response bằng một wrapper `ApiResponse<T>` cho các API thành công, gồm các field như `success`, `code`, `message`, `data`, `traceId` và `timestamp`. Mục tiêu là để frontend/mobile có một contract ổn định, để parse và để xử lý business code.

Với các API dạng list hoặc search, em không trả trực tiếp `Page` của Spring Data vì object này chứa nhiều thông tin không cần expose. Em map sang `PagedResponse<T>` gồm `items` và `page metadata` như `page`, `size`, `totalElements`, `totalPages`, `hasNext`, `hasPrevious`.

Với lỗi, em không dùng chung `ApiResponse` một cách tùy tiện mà thiết kế `ErrorResponse` riêng, kết hợp với `GlobalExceptionHandler`. Error response có `status`, `error`, `code`, `message`, `path`, `traceId`, `timestamp`, và nếu là validation error thì có thêm `details` cho từng field lỗi.

Một điểm quan trọng là response body chỉ là một phần, còn HTTP status vẫn phải đúng. Ví dụ create thành công trả `201`, validation fail trả `400` hoặc `422`, not found trả `404`, conflict trả `409`, internal error trả `500`. Không nên trả HTTP 200 cho mọi lỗi vì sẽ làm sai semantics của HTTP và gây khó cho gateway, monitoring hoặc client SDK.

Ngoài ra, em không wrap tất cả response một cách máy móc. Những API như file download, streaming, actuator health check hoặc API public theo contract đối tác thì nên giữ response riêng. Với các API business thông thường, dùng `ApiResponse<T>` giúp hệ thống thống nhất, dễ maintain và dễ debug production nhờ `traceId`.

31. Template khuyến nghị cuối cùng

Success response:

```
{
  "success": true,
  "code": "SUCCESS",
  "message": "Request processed successfully",
  "data": {},
  "traceId": "c5a1f2e9d7b84a3d",
  "timestamp": "2026-06-04T10:00:00Z"
}
```

Paged response:

```
{
  "success": true,
  "code": "SUCCESS",
  "message": "Request processed successfully",
  "data": {
    "items": [],
    "page": {
      "page": 0,
      "size": 20,
      "totalElements": 100,
    }
  }
}
```

```
    "totalPages": 5,
    "first": true,
    "last": false,
    "hasNext": true,
    "hasPrevious": false
  }
},
"traceId": "c5a1f2e9d7b84a3d",
"timestamp": "2026-06-04T10:00:00Z"
}
```

Error response:

```
{
  "success": false,
  "timestamp": "2026-06-04T10:00:00Z",
  "status": 400,
  "error": "BAD_REQUEST",
  "code": "ORDER_001",
  "message": "Order has already been paid",
  "path": "/api/orders/1001/pay",
  "traceId": "c5a1f2e9d7b84a3d"
}
```

Validation error response:

```
{
  "success": false,
  "timestamp": "2026-06-04T10:00:00Z",
  "status": 422,
  "error": "UNPROCESSABLE_ENTITY",
  "code": "COMMON_422",
  "message": "Validation failed",
  "path": "/api/orders",
  "traceId": "c5a1f2e9d7b84a3d",
  "details": [
    {
      "field": "quantity",
      "message": "must be greater than 0",
      "rejectedValue": -1
    }
  ]
}
```

32. Kết luận

Một thiết kế ApiResponse chuẩn production nên có:

- `ApiResponse<T>` cho success.
- `ErrorResponse` cho failure.
- `PagedResponse<T>` cho phân trang.
- `traceId` cho debug.
- `code` cho business handling.
- `message` an toàn cho client.
- HTTP status đúng chuẩn.
- Không trả entity trực tiếp.
- Không wrap file download/streaming/actuator.
- Không trả HTTP 200 cho mọi lỗi.

Cách thiết kế này giúp API contract rõ ràng, frontend dễ xử lý, backend dễ maintain, QA dễ test và production team dễ debug khi có incident.